

Лабораторная работа №8. Адаптивная верстка: Grid

Цель работы: освоить современные технологии CSS-разметки — Grid.

Адаптивная верстка — это подход к созданию веб-страниц, при котором дизайн и контент автоматически подстраиваются под размер экрана устройства (компьютер, планшет, смартфон).

Grid — двумерная система компоновки для создания сложных сеток.

Шаг 1. Создание рабочей директории

1. Откройте **Visual Studio Code**. Выберите **File** → **New Window** (**Ctrl + Shift + N**).
2. Нажмите **Ctrl + `** чтобы открыть терминал VS Code.
3. Перейдите в папку с документами:

cd Documents/

4. Создайте директорию для лабораторной работы:

mkdir Lab8_Adaptive_FIO (где *FIO* — ваша фамилия)

5. Перейдите в созданную директорию:

cd Lab8_Adaptive_FIO

6. Создайте файлы:

touch index.html grid.css README.md

- **index.html** — для HTML-разметки
- **grid.css** — для CSS-стилей
- **README.md** — для описания

7. Создайте папку для скриншотов:

mkdir img

Шаг 2. Подключение рабочего каталога в VS Code

1. Откройте **Explorer** (**Ctrl + B**).
2. Нажмите **Open Folder**.
3. Выберите папку **Lab8_Adaptive_FIO** и откройте её.

Шаг 3. Инициализация Git-репозитория

1. Инициализируйте репозиторий:

git init -b main

2. Добавьте файлы в индекс:

git add .

3. Создайте первый коммит:

git commit -m "Initial commit"

Шаг 4. Подключение к GitHub

1. Создайте новый **public**-репозиторий на GitHub:

Lab8_AdaptiveGrid

Без **README**, **.gitignore** и **License**.

2. Привяжите локальный репозиторий:

git remote add origin <URL_вашего_репозитория>

git push -u origin main

Практическое задание

1. Сделайте скриншот **Git Bash** с выполненными командами подключения и пуша.

2. Сохраните файл в папку **img**:

gitPushLab8_FIO.png

3. Выполните в терминале:

git add .

git commit -m "Added push screenshot"

git push

Шаг 5. Практика с CSS Grid

Теория. Введение в Grid

CSS Grid — это мощная двумерная система компоновки, которая позволяет создавать сложные сетки по строкам и столбцам.

Grid идеально подходит для:

- Создания полноценных макетов страниц
- Сложных раскладок с несколькими областями
- Галерей изображений
- Дашбордов и административных панелей

Основные концепции

Grid-контейнер — родительский элемент с `display: grid`

Grid-элементы — дочерние элементы внутри контейнера

Grid-линии — разделительные линии между ячейками

Grid-ячейки — пересечение строки и столбца

Grid-области — прямоугольные области из нескольких ячеек

5.1 Подготовка HTML-файла

Шаг 5.1.1 Создание нового файла

1. Откройте **index.html** в редакторе
2. Введите **Emmet-команду !** и нажмите **Enter**

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Document</title>
  </head>
  <body></body>
</html>
```

Emmet автоматически создаёт структуру HTML

Шаг 5.1.2 Исправление базовых параметров

Измените код:

```
<html lang="ru-RU">
```

```
<title>CSS Grid</title>
```

Подключите CSS-файл (Emmet: **link:css**):

```
<link rel="stylesheet" href="grid.css" />
```

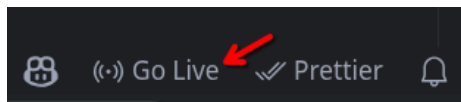
5.2 Базовая структура Grid

Шаг 5.2.1 HTML-разметка сетки

Внутри **<body>** добавьте заголовок и контейнер:

```
<body>
  <h1>Примеры CSS Grid</h1>
  <div class="grid-container"></div>
</body>
```

Запустите вашу страницу через **Live Server**, чтобы наблюдать изменения:



В **контейнере** введите:

.grid-item*6

Нажмите **TAB**:

```
<div class="grid-container">
  .grid-item*6 TAB
</div>
```

Отформатируем немного вручную (добавим цифры):

```
<div class="grid-container">
  <div class="grid-item">1</div>
  <div class="grid-item">2</div>
  <div class="grid-item">3</div>
  <div class="grid-item">4</div>
  <div class="grid-item">5</div>
  <div class="grid-item">6</div>
</div>
/body>
```

Результат:

Элементы отображаются обычным столбцом — Grid ещё не включён.

Примеры CSS Grid

1
2
3
4
5
6

5.3 Создание Grid-контейнера

Шаг 5.3.1 Подключение Grid

В файл **grid.css** добавьте:

```
.grid-container {
  display: grid;
}
```

Результат:

Визуально почти ничего не изменилось, но контейнер стал Grid-контейнером.

5.4 Столбцы Grid

Шаг 5.4.1 Явное задание столбцов

Добавьте в **.grid-container**:

```
.grid-container {
  display: grid;
  + grid-template-columns: 200px 200px 200px;
}
```

Результат:

Созданы три столбца по 200px, элементы распределились по ним.

Примеры CSS Grid

1
4

2
5

3
6

Шаг 5.4.2 Единица измерения fr

Замените свойство:

```
.grid-container {  
  display: grid;  
  grid-template-columns: 1fr 2fr 1fr;  
}
```

Результат:

Средний столбец стал в 2 раза шире остальных.

Примеры CSS Grid

1
4

2
5

3
6

Единица fr (fraction) — доля свободного пространства

Шаг 5.4.3 Функция repeat(). Повторение значений

Замените строку:

```
.grid-container {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
}
```

Результат:

Три равных по ширине столбца.

Примеры CSS Grid

1
4

2
5

3
6

5.5 Строки Grid

5.5.1 Задание строк

Добавьте:

```
.grid-container {  
  display: grid;  
  grid-template-rows: 100px auto auto auto auto 100px;  
}
```

Результат:

- Первая и последняя строки фиксированной высоты
- Центральная строка подстраивается под контент

Примеры CSS Grid



5.6 Отступы между ячейками

5.6.1 Общее расстояние

Добавьте:

```
.grid-container {  
  display: grid;  
  gap: 20px;  
}
```

Результат:

Между всеми ячейками появились отступы.

Примеры CSS Grid

1
2
3
4
5
6

5.7 Именованные области Grid

5.7.1 Разметка областей

Замените содержимое `.grid-container` в HTML:

```
<div class="grid-container">  
  <header class="header">Header</header>  
  <aside class="sidebar">Sidebar</aside>  
  <main class="main">Main</main>  
  <footer class="footer">Footer</footer>  
</div>  
</body>
```

5.7.2 Описание областей в CSS

Обновите **grid.css**:

```
.grid-container {  
  display: grid;  
  grid-template-columns: 1fr 2fr 1fr;  
  grid-template-areas:  
    "header header header"  
    "sidebar main main"  
    "footer footer footer";  
  gap: 20px;  
}  
.header {  
  grid-area: header;  
}  
.sidebar {  
  grid-area: sidebar;  
}  
.main {  
  grid-area: main;  
}  
.footer {  
  grid-area: footer;  
}
```

Определяем именованные области сетки:

- **Первая строка:** header занимает все 3 колонки
- **Вторая строка:** sidebar в первой колонке, main во второй и третьей колонках
- **Третья строка:** footer занимает все 3 колонки

Результат:

Элементы расположены по именованным областям.

Примеры CSS Grid

Header

Sidebar

Main

Footer

5.8 Выравнивание элементов

5.8.1 Выравнивание внутри ячеек

Добавим выравнивание элементов по центру внутри их ячеек (горизонтально и вертикально):

```
.grid-container {  
  display: grid;  
  grid-template-columns: 1fr 2fr 1fr;  
  grid-template-areas:  
    "header header header"  
    "sidebar main main"  
    "footer footer footer";  
  gap: 20px;  
  justify-items: center;  
  align-items: center;  
}
```

Результат:

Контент центрируется внутри каждой ячейки.

Примеры CSS Grid



Практическое задание

1. Сделайте скриншот страницы в браузере.
2. Сохраните изображения в папку **img**:

grid_examplesGrid_Lab8_FIO.png

3. Выполните в **терминале**:

git add .

git commit -m "Added Grid practice"

git push

Шаг 6. Практика с CSS Grid

В этом шаге мы:

- закрепим навыки работы с **CSS Grid**
- создадим несколько типовых сеток

6.1 Создание файлов проекта

Создайте через терминал файлы **gridPractice.html** и **gridPractice.css**

6.1.1 Создание HTML-файла

1. Создайте файл **gridPractice.html**
2. Откройте его в редакторе
3. Введите Emmet-команду **!** и нажмите **Enter**

6.1.2 Базовые изменения HTML

Измените автоматически созданный код:


```

<!DOCTYPE html>
<html lang="ru-RU">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
    <title>Grid Examples</title>
    <link rel="stylesheet" href="gridPractice.css" />
  </head>
  <body></body>
</html>

```

6.1.3 Заголовок страницы

Внутри <body> добавьте:

```

<body>
+ <h1>Примеры CSS Grid</h1>
</body>

```

Действие: Запустите **gridPractice.html** через **Live Server**.

Пока отображается только заголовок:

Примеры CSS Grid

Сделайте коммит:

git add .

git commit -m "Added base HTML structure for Grid practice"

6.2 Пример 1. Простая Grid-сетка

6.2.1 HTML-разметка сетки (Emmet)

Добавьте секцию:

```

<body>
  <h1>Примеры CSS Grid</h1>
  <section>
    <h2>Пример 1: Сетка 3x3</h2>
    + <div class="simple-grid"></div>
  </section>
</body>

```

Внутри **.simple-grid** используйте **Emmet**:

div.grid-item*9

```
<div class="simple-grid">
  div.grid-item*9| TAB
</div>
```

Заполните элементы числами от 1 до 9.

```
<div class="simple-grid">
  <div class="grid-item">1</div>
  <div class="grid-item">2</div>
  <div class="grid-item">3</div>
  <div class="grid-item">4</div>
  <div class="grid-item">5</div>
  <div class="grid-item">6</div>
  <div class="grid-item">7</div>
  <div class="grid-item">8</div>
  <div class="grid-item">9</div>
</div>
```

Шаг 6.2.2 CSS для простой сетки

Создайте и откройте файл **gridPractice.css**. Куда добавьте:

```
.simple-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
}
```

Результат:

У нас появилась сетка 3x3:

Примеры CSS Grid

Пример 1: Сетка 3x3

1	2	3
4	5	6
7	8	9

Теперь сделаем её визуально более привлекательной:

```
.grid-item {
  background-color: green;
  color: white;
  padding: 40px;
  text-align: center;
  font-size: 24px;
  border-radius: 5px;
}
```

Результат:

Примеры CSS Grid

Пример 1: Сетка 3x3

1	2	3
4	5	6
7	8	9

Сделайте коммит:

git add .

git commit -m "Added simple 3x3 Grid example"

6.3 Пример 2. Макет страницы на Grid

6.3.1 HTML-структура макета

Добавьте следующую секцию в **gridPractice.html**:

```
<body>
  <h1>Примеры CSS Grid</h1>
  <!-- Пример 1: Простая сетка -->
  <section>|...
</section>
  <!-- Пример 2: Макет страницы -->
  <section>
    <h2>Пример 2: Макет страницы</h2>
    <div class="page-layout">
      <header class="header">Header</header>
      <aside class="sidebar">Sidebar</aside>
      <main class="main">Main Content</main>
      <footer class="footer">Footer</footer>
    </div>
  </section>
</body>
```

6.3.2 Grid-области в CSS

Добавьте в **gridPractice.css**:

```
.page-layout {
  display: grid;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
  grid-template-columns: 200px 1fr;
  grid-template-rows: 80px 1fr 60px;
  gap: 10px;
  height: 500px;
}
```

Создает сетку 3×2 с фиксированными размерами:

- **2 колонки:** 200px (боковая панель) + оставшееся пространство
- **3 строки:** 80px (шапка) + оставшееся пространство + 60px (подвал)
- **Высота контейнера:** 500px
- **Зазоры:** 10px между элементами
- **Области:** шапка и подвал занимают всю ширину, боковая панель слева, основной контент справа

```
.header {  
  grid-area: header;  
  background-color: #333;  
  color: white;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  border-radius: 5px;  
}
```

Занимает область header (верхняя строка, обе колонки)

- Темный фон (#333) с белым текстом
- Текст выровнен по центру (горизонтально и вертикально)
- Скругленные углы 5px

```
.sidebar {  
  grid-area: sidebar;  
  background-color: #f0f0f0;  
  padding: 20px;  
  border-radius: 5px;  
}
```

Занимает область sidebar (вторая строка, первая колонка)

- Светло-серый фон (#f0f0f0)
- Внутренние отступы 20px
- Скругленные углы 5px

```
.main {  
  grid-area: main;  
  background-color: #fff;  
  border: 2px solid #ddd;  
  padding: 20px;  
  border-radius: 5px;  
}
```

Занимает область main (вторая строка, вторая колонка)

- Белый фон
- Рамка 2px серого цвета (#ddd)
- Внутренние отступы 20px
- Скругленные углы 5px

```
.footer {  
  grid-area: footer;  
  background-color: #333;  
  color: white;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  border-radius: 5px;  
}
```

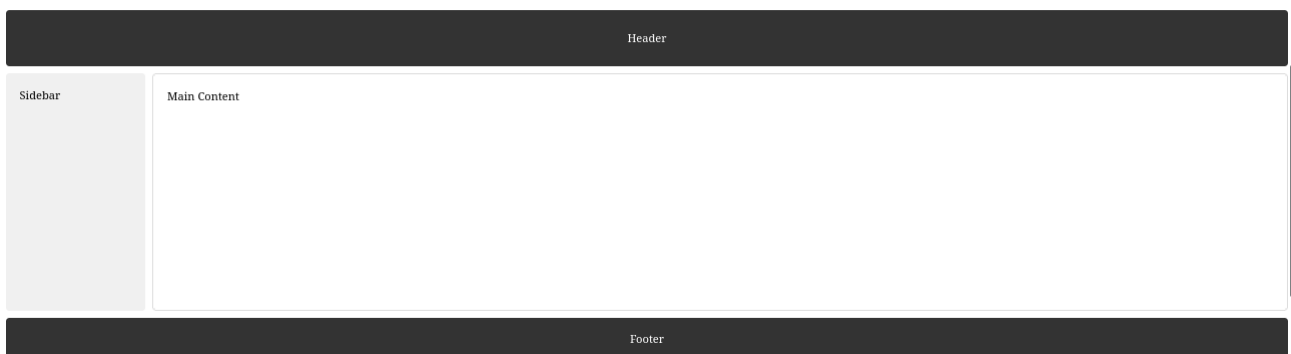
Занимает область footer (нижняя строка, обе колонки)

- Темный фон (#333) с белым текстом
- Текст выровнен по центру (горизонтально и вертикально)
- Скругленные углы 5px

Результат:

Макет страницы чётко разделён на области.

Пример 2: Макет страницы



Сделайте коммиты:

git add .

git commit -m "Added page layout using grid-template-areas"

6.4 Пример 3. Адаптивная галерея

6.4.1 HTML-разметка галереи (Emmet)

Добавьте секцию:

```

27 > ... <section> ...
35 ... </section>
36 ... <!-- Пример 3: Галерея -->
37 ... <section>
38 ... <h2>Пример 3: Адаптивная галерея</h2>
39 ... + <div class="gallery"></div>
40 ... </section>
41 ... </body>

```

Внутри **.gallery** используйте Emmet:

div.gallery-item*6

```

<div class="gallery">
  div.gallery-item*6 TAB
</div>

```

Добавьте текст «Фото 1» – «Фото 6»:

```

<div class="gallery">
  <div class="gallery-item">Фото 1</div>
  <div class="gallery-item">Фото 2</div>
  <div class="gallery-item">Фото 3</div>
  <div class="gallery-item">Фото 4</div>
  <div class="gallery-item">Фото 5</div>
  <div class="gallery-item">Фото 6</div>
</div>

```

Результат:

Пример 3: Адаптивная галерея

Фото 1
 Фото 2
 Фото 3
 Фото 4
 Фото 5
 Фото 6

6.4.2 CSS для адаптивной галереи

Добавьте в **gridPractice.css**:

```

.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 15px;
}

```

Создает адаптивную сетку для галереи

- **repeat(auto-fit, minmax(200px, 1fr))**: автоматически подстраивает количество колонок — каждая колонка минимум 200px, но может растягиваться на доступное пространство; при уменьшении экрана колонки переносятся на новую строку

- **Зазоры:** 15px между элементами

```
.gallery-item {  
  background-color: #2196f3;  
  color: white;  
  padding: 60px;  
  text-align: center;  
  border-radius: 5px;  
  transition: transform 0.3s;  
}
```

Стиль для каждого элемента галереи (карточки)

- Синий фон (#2196f3) с белым текстом
- Внутренние отступы 60px (квадратные карточки)
- Текст выровнен по центру
- Скругленные углы 5px
- Плавная анимация при наведении (подготовка через transition)

```
.gallery-item:hover {  
  transform: scale(1.05);  
}
```

Эффект при наведении курсора: карточка увеличивается на 5% (scale(1.05))

- Анимация длится 0.3 секунды (задана в transition)

Результат:

Галерея автоматически подстраивается под ширину экрана.

Пример 3: Адаптивная галерея



Сделайте коммит:

git add .

git commit -m "Added responsive grid gallery"

Практическое задание

1. Сделайте скриншот страницы в браузере со всеми примерами Grid.
2. Сохраните изображение в папку **img**:

grid_examplesGridPractice_Lab8_FIO.png

3. Выполните в терминале:

git add .

git commit -m "Added Grid practice"

git push

Самостоятельное задание №1. Оформление README.md

1. Создайте подробное описание лабораторной работы в файле **README.md**.

Вставьте также следующую таблицу:

Сравнение Flexbox и CSS Grid

Общая идея

Критерий	Flexbox	CSS Grid
Тип компоновки	Одномерная	Двумерная
Управление	Строка или столбец	Строки и столбцы
Основное назначение	Выравнивание элементов	Построение макетов
Подходит для	Компонентов	Страниц и сеток

Вывод: Flexbox лучше подходит для **линейных структур**, Grid — для **табличных и сложных**.

Типичные сценарии использования

Задача	Рекомендуется
Навигационное меню	Flexbox
Карточки товаров	Flexbox + Grid
Центрирование элемента	Flexbox
Макет страницы	Grid
Галерея изображений	Grid
Дашборд	Grid

Выполните в терминале:

git add .

git commit -m "Added complete README for Lab8"

git push

Самостоятельное задание №2: Создание Grid-макета

1. Создайте файл **grid-practice.html** и реализуйте дашборд с такими **областями**:

- **Header** — на всю ширину
- **Sidebar** — левая колонка (200px)
- **Main content*** — основная область (занимает оставшееся пространство)
- **Widget 1** — правая колонка верхняя (300px ширина, 200px высота)
- **Widget 2** — правая колонка нижняя (300px ширина, 200px высота)
- **Footer** — на всю ширину

2. Требования:

- Используйте *grid-template-areas*
- Высота **header**: 80px, **footer**: 60px
- Общая **высота**: минимум 600px
- Добавьте **gap**: 15px

3. Сделайте **скриншот** и сохраните в **img**:

gridPracticeLab8_FIO.png

4. Выполните в **терминале**:

git add .

git commit -m "Added Grid practice"

git push